

Evaluación (1ª instancia)

Duración: 3 horas

Ejercicio 0: Pregunta individual sobre el laboratorio

Describe la forma en que implementó el tablero, indicando si utiliza lista de listas o matriz de funtores.

Ejercicio 1 [30 puntos]

Denominaremos descomposición cuadrada de un entero positivo N a un conjunto de números diferentes $[H_1, H_2, \dots, H_k]$, tales que: $H_1^2 + H_2^2 + \dots + H_k^2 = N$

Por ejemplo: $[1, 3]$ es una descomposición cuadrada de 10 ya que $1^2 + 3^2 = 1 + 9 = 10$.

Notar que no siempre existe la descomposición cuadrada de un número, y cuando existe no tiene por qué ser única.

Implementar en Prolog los siguientes predicados que nos permitan encontrar estas descomposiciones:

a) `suma_cuadrados(+L, ?N)` $\leftarrow$ L es una lista de números únicos, N es el resultado de sumar el cuadrado de todos esos números. Ejemplo:

```
?- suma_cuadrados([2,3],13).
```

b) `cuadrados_menores(+N, ?L)` $\leftarrow$ N es un entero positivo, L es una lista de todos los enteros positivos que tengan un cuadrado menor o igual a N . Ejemplo:

```
?- cuadrados_menores(60,[1,2,3,4,5,6,7]).
```

c) `descomposicion_cuadrada(+N, ?L)` $\leftarrow$ N es un entero positivo, L es una descomposición cuadrada de N como la definida anteriormente. Ejemplos:

```
?- descomposicion_cuadrada(30,L).
```

```
L = [1, 2, 3, 4] ;
```

```
L = [1, 2, 5]
```

```
?- descomposicion_cuadrada(31,L).
```

```
false.
```

d) `descomposicion_minima(+N, ?L)` $\leftarrow$ N es un entero positivo, L es la descomposición cuadrada de N cuya *suma de valores* (sin elevar al cuadrado) es la más baja posible. Ejemplo:

```
?- descomposicion_minima(30,?L)
```

```
L = [1, 2, 5]
```

Ejercicio 2 [15 puntos]

a) Indique si son verdaderas o falsas las siguientes afirmaciones. Fundamente.

i) Si dos expresiones son unificables, existe un único m.g.u. entre ellas.

ii) Una cláusula de Horn es un tipo particular de cláusula definida.

iii) Dados un programa lógico y un objetivo, diferentes reglas de computación pueden generar diferentes árboles SLD.

iv) Una sustitución θ unifica un conjunto de expresiones W sii $(W\theta)$ es un conjunto no vacío.

b) Completar la siguiente definición de resolvente binaria para cláusulas de primer orden:

Sean $C1$ y $C2$ dos cláusulas de primer orden, sin variables en común:

$C1 : \{L1, \dots, Ln\}$

$C2 : \{M1, \dots, Mk\}$

y θ un mgu de Li y $\neg Mj$

*La resolvente binaria es la siguiente cláusula $C3$: **COMPLETAR***

Ejercicio 3 [25 puntos]

Considere el siguiente programa lógico:

```
f(X,Y) :- g(X,Y).
f(0,0).
g(X,Y) :- p(X), q(Y).
g(X,Y) :- f(X,Y).
p(1).
p(2).
q(1).
q(2).
q(3).
```

- i) Escriba el árbol SLD resultante de hacer la consulta $\leftarrow f(X, Y)$
- ii) ¿Qué respuestas devuelve el intérprete prolog y en qué orden?
- iii) Muestre cómo cambia el árbol si cambiamos la regla $g(X, Y) :- p(X), q(Y)$ por las siguientes variantes:
 - a) $g(X, Y) :- !, p(X), q(Y).$
 - b) $g(X, Y) :- p(X), !, q(Y).$
 - c) $g(X, Y) :- p(X), q(Y), !.$

Ejercicio 4 [15 puntos]

a) Escriba una gramática lógica que reconozca expresiones aritméticas simples. Las expresiones posibles son:

- número: dígito del 0 al 9 o dígito negado.
- operador binario aplicado a dos números.

Los operadores binarios son: *mas*, *menos*, *por*, *dividido*. El operador de negación es *neg*.

Ejemplos de expresiones válidas:

```
[5]
[neg, 3]
[3, por, 2]
[1, mas, neg, 3]
```

b) Escriba una gramática lógica que reconozca las expresiones de la parte a) y las evalúe.

Asuma definidos los siguientes predicados:

```
suma(N1,N2,N3) :- N3 is N1 + N2.
diferencia(N1,N2,N3) :- N3 is N1 - N2.
producto(N1,N2,N3) :- N3 is N1 * N2.
cociente(N1,N2,N3) :- N3 is N1 / N2.
negativo(N1,N2) :- N2 is -N1.
```

Sugerencia: utilizar los predicados `=..` y `call`, que permiten crear e invocar predicados.

Ejemplos de consultas:

```
?- s(Res, [1, mas, 2], []).
Res = 3.
?- s(-4, [2, por, neg, 2], []).
true.
```